

```
/*  
JORGE MEJIAS DONOSO - ALBERTO MENCHEN MONTERO - 206  
100495807@alumons.uc3m.es 100495692@alumnos.uc3m.es  
*/
```

```
%{          // SECCION 1 Declaraciones de C-Yacc
```

```
#include <stdio.h>
```

```
#include <ctype.h>    // declaraciones para tolower
```

```
#include <string.h>    // declaraciones para cadenas
```

```
#include <stdlib.h>    // declaraciones para exit ()
```

```
#define FF fflush(stdout); // para forzar la impresion inmediata
```

```
int yylex ();
```

```
int yyerror ();
```

```
char *mi_malloc (int);
```

```
char *gen_code (char *);
```

```
char *int_to_string (int);
```

```
char *char_to_string(char);
```

```
char temp [4096]; // variable para almacenar cadenas temporales
```

```
char *funcion_actual = NULL; // Nombre de la función actual
```

```
// Abstract Syntax Tree (AST) Node Structure
```

```
typedef struct ASTnode t_node;
```

```
struct ASTnode{  
    char *op;  
    int type;           // leaf, unary or binary nodes  
    t_node *left;  
    t_node *right;  
};
```

```
// Definitions for explicit attributes
```

```
typedef struct s_attr {  
    int value ; // - Numeric value of a NUMBER  
  
    char *code ; // - to pass IDENTIFIER names, and other translations  
  
    t_node *node ; // - for possible future use of AST  
} t_attr ;
```

```
#define YYSTYPE t_attr
```

```
// Tabla de variables locales
```

```
typedef struct variable_local {  
    char *nombre;  
  
    struct variable_local *siguiente;  
} tipo_variable_local;
```

```
tipo_variable_local *variables_locales = NULL;
```

```
int es_variable_local(char *nombre) {  
    tipo_variable_local *actual = variables_locales;
```

```
while (actual != NULL) {  
    if (strcmp(actual->nombre, nombre) == 0) {  
        return 1;  
    }  
    actual = actual->siguiente;  
}  
return 0;  
}
```

```
void anadir_var_local(char *nombre) {  
    tipo_variable_local *variable_nueva = (tipo_variable_local *)mi_malloc(sizeof(tipo_variable_local));  
    variable_nueva->nombre = gen_code(nombre);  
    variable_nueva->siguiente = variables_locales;  
    variables_locales = variable_nueva;  
}
```

```
void vaciar_variables_locales() {  
    variables_locales = NULL;
```

```
}
```

```
%}
```

```
// Definitions for explicit attributes
```

```
%token NUMBER
```

```
%token IDENTIF          // Identificador=variable
```

```
%token INTEGER          // identifica el tipo entero
```

```
%token STRING           // identifica el tipo string
```

```
%token MAIN             // identifica el comienzo del proc. main
```

```
%token WHILE            // identifica el bucle main
```

```
%token IF               // identifica la sentencia if
```

```
%token ELSE            // identifica la sentencia else
```

```
%token PUTS            // identifica la sentencia puts
```

```
%token PRINTF          // identifica la sentencia printf
```

```
%token IGUAL DISTINTO MENORIGUAL MAYORIGUAL // operadores de comparación
```

```
%token AND OR          // operadores lógicos
```

```
%token FOR             // identifica la sentencia for
```

```

%token RETURN                // identifica la sentencia return

%right '='                   // asignación

%left OR AND NOT             // or and not lógicos

%left IGUAL DISTINTO         // comparación de igualdad y desigualdad

%left '<' '>' MENORIGUAL MAYORIGUAL // comparación numérica

%left '+' '-'                // suma y resta

%left '*' '/' '%'            // multiplicación, división y modulo

%left UNARY_SIGN             // signo unario


%%                            // Seccion 3 Gramatica - Semantico


//*****/

// axioma

programa:

    vars_globales conjunto_funciones {

```

```
    printf("%s\n%s\n", $1.code, $2.code);  
}  
;
```

// Declaraciones de variables globales

vars_globales:

```
/* vacío */ {  
    $$code = gen_code("");  
}  
| vars_globales definicion_var_global ';' {  
    sprintf(temp, "%s\n%s", $1.code, $2.code);  
    $$code = gen_code(temp);  
}  
;
```

// Definición de variables globales

definicion_var_global:

```
INTEGER conjunto_vars_global {  
    $$code = $2.code;
```

```
}  
;
```

// Definición de un conjunto de variables globales

conjunto_vars_global:

```
IDENTIF {  
    sprintf(temp, "(setq %s 0)", $1.code);  
    $$code = gen_code(temp);  
}  
| IDENTIF '=' expresion {  
    sprintf(temp, "(setq %s %d)",  
        $1.code, $3.value); $$code = gen_code(temp);  
}  
| conjunto_vars_global ',' IDENTIF {  
    sprintf(temp, "%s\n(setq %s 0)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
| conjunto_vars_global ',' IDENTIF '=' expresion {  
    sprintf(temp, "%s\n(setq %s %d)", $1.code, $3.code, $5.value);
```



```

    $$code = gen_code(temp);
}
| IDENTIF '[' expresion ']' {
    sprintf(temp, "(setq %s (make-array %d))", $1.code, $3.value);
    $$code = gen_code(temp);
}
| conjunto_vars_global ';' IDENTIF '[' expresion ']' {
    sprintf(temp, "%s\n(setq %s (make-array %d))", $1.code, $3.code, $5.value);
    $$code = gen_code(temp);
}
}
;

```

// Definición de funciones

conjunto_funciones:

```

funciones_con_main {
    $$code = $1.code;
}
;

```

// Definición de funciones con la función principal

funciones_con_main:

```
funcion_principal {  
    $$code = $1.code;  
}  
| funciones_auxiliares funcion_principal {  
    sprintf(temp, "%s\n%s", $1.code, $2.code);  
    $$code = gen_code(temp);  
}  
;
```

// Definición de funciones auxiliares

funciones_auxiliares:

```
definicion_funcion {  
    $$code = $1.code;  
}  
| funciones_auxiliares definicion_funcion {  
    sprintf(temp, "%s\n%s", $1.code, $2.code);  
    $$code = gen_code(temp);  
}
```

```
}  
;
```

// Definición de una función

definicion_funcion:

```
cabecera_funcion '{ bloque_sentencias }' {  
    // Aquí ya funcion_actual está bien definida  
    char *ultima_instruccion = strrchr($3.code, '\n');  
    if (ultima_instruccion != NULL) {  
        ultima_instruccion += 1;  
    } else {  
        ultima_instruccion = $3.code;  
    }  
    // Verificamos si la última instrucción es un return  
    if (strstr(ultima_instruccion, "return") != NULL) {  
        sprintf(temp, "(defun %s (%s)\n%s)", funcion_actual, $1.code, $3.code);  
    } else {  
        sprintf(temp, "(defun %s (%s)\n%s\n)", funcion_actual, $1.code, $3.code);  
    }  
}
```

```
    $$code = gen_code(temp);  
    free(funcion_actual);  
}  
;
```

// Definición de la cabecera de la función

cabecera_funcion:

```
    IDENTIF '(' argumentos_funcion ')' {  
        funcion_actual = strdup($1.code);  
        sprintf(temp, "%s", $3.code);  
        $$code = gen_code(temp);  
  
    }  
;
```

// Definición de los argumentos de la función

argumentos_funcion:

```
    /* vacío */ {
```

```
    $$code = gen_code("");  
  }  
  | conjunto_parametros {  
    $$code = $1.code;  
  }  
  ;
```

// Definición de los parámetros de la función

conjunto_parametros:

```
  entrada_funcion {  
    $$code = $1.code;  
  }  
  | conjunto_parametros ';' entrada_funcion {  
    sprintf(temp, "%s %s", $1.code, $3.code);  
    $$code = gen_code(temp);  
  }  
  ;
```

// Definición de un parámetro de la función

entrada_funcion:

```
INTEGER IDENTIF {  
    sprintf(temp, "%s", $2.code);  
    $$code = gen_code(temp);  
}  
;
```

// Definición de la función principal

inicio_main:

```
MAIN '(' ')' {  
    funcion_actual = strdup("main");  
    // Definimos la función principal  
    vaciar_variables_locales();  
}  
;
```

// Definición del bloque principal de la función

funcion_principal:

```
inicio_main '{' bloque_sentencias '}' {  
    sprintf(temp, "(defun main ()\n%s\n)", $3.code);  
    $$code = gen_code(temp);  
    // Liberar la memoria de la variable global  
    vaciar_variables_locales();  
    free(funcion_actual);  
}  
;
```

// Definición del bloque de sentencias

bloque_sentencias:

```
sentencia_sep {  
    $$ = $1;  
}  
| bloque_sentencias sentencia_sep {  
    sprintf(temp, "%s\n%s", $1.code, $2.code);  
    $$code = gen_code(temp);  
}
```

```
;
```

```
// Definición de una sentencia
```

```
sentencia_sep:
```

```
    sentencia_simple ';' {
```

```
        $$ = $1;
```

```
    }
```

```
| sentencia_bloque {
```

```
    $$ = $1;
```

```
}
```

```
| declaracion_variable ';' {
```

```
    $$ = $1;
```

```
}
```

```
;
```

```
// Definición de una sentencia simple
```

```
sentencia_simple:
```

```
    IDENTIF '=' expresion { // Asignación a una variable
```

```
        if(!es_variable_local($1.code)){
```



```

    sprintf(temp, "(setf %s %s)", $1.code, $3.code);
} else {
    sprintf(temp, "(setf %s_%s %s)", funcion_actual, $1.code, $3.code);
}
$$code = gen_code(temp);
}

| IDENTIF '[' expresion ']' '=' expresion { // Asignación a un elemento de un array
    if(!es_variable_local($1.code)) {
        sprintf(temp, "(setf (aref %s %s) %s)", $1.code, $3.code, $6.code);
    } else {
        sprintf(temp, "(setf (aref %s_%s %s) %s)", funcion_actual, $1.code, $3.code, $6.code);
    }
    $$code = gen_code(temp);
}

| IDENTIF '(' argumentos ')' { // Llamada a función como sentencia y con posibles argumentos
    sprintf(temp, "(%s %s)", $1.code, $3.code);
    $$code = gen_code(temp);
}

| '@' expresion {

```

```

        sprintf(temp, "(print %s)", $2.code);
        $$code = gen_code(temp);
    }
| PUTS '(' STRING ')' { // traduccion de puts
        sprintf(temp, "(print \"%s\\\")", $3.code);
        $$code = gen_code(temp);
    }
| PRINTF '(' STRING ',' lista_elementos ')' { //traduccion de printf
        $$code = $5.code;
    }
| RETURN expresion {
    if (funcion_actual != NULL) {
        if (!es_variable_local($2.code)) { //comprobamos si es variable local
            sprintf(temp, "(return-from %s %s)", funcion_actual, $2.code);
        } else {
            sprintf(temp, "(return_from %s %s_%s)", funcion_actual, funcion_actual, $2.code);
        }
    }
}
$$code = gen_code(temp);

```

```
}  
;
```

// Definición de una sentencia de control

sentencia_bloque:

```
    if_solo  
    | if_else  
    | WHILE '(' expresion ')' '{' bloque_sentencias '}' { // Definición de la sentencia while  
        sprintf(temp, "(loop while %s do\n%s)", $3.code, $6.code);  
        $$code = gen_code(temp);  
    }  
    | FOR '(' asignacion_inicial ';' expresion ';' aumento ')' '{' bloque_sentencias '}' {  
        // Definición del bucle for  
        sprintf(temp, "%s\n(loop while %s do\n%s\n%s)",  
            $3.code, $5.code, $10.code, $7.code);  
        $$code = gen_code(temp);  
    }  
;
```

// definicion del aumento bucle for

aumento:

```
IDENTIF '=' expresion {  
    if (!es_variable_local($1.code)) { //comprobamos si es variable local  
        sprintf(temp, "(setf %s %s)", $1.code, $3.code);  
    } else {  
        sprintf(temp, "(setf %s_%s %s)", funcion_actual, $1.code, $3.code);  
    }  
    $$code = gen_code(temp);  
}  
;
```

// Definición de la asignación inicial del bucle for

asignacion_inicial:

```
IDENTIF '=' operando {  
    if (!es_variable_local($1.code)) { //comprobamos si es variable local  
        sprintf(temp, "(setq %s %s)", $1.code, $3.code);  
    } else {
```

```
    sprintf(temp, "(setq %s_%s %s)", funcion_actual, $1.code, $3.code);  
  }  
  $$code = gen_code(temp);  
}  
;
```

// Definición de la sentencia if-else

if_else:

```
IF '(' expresion ')' '{' bloque_sentencias '}' ELSE '{' bloque_sentencias '}' {  
    sprintf(temp, "(if %s\n(progn\n%s)\n(progn\n%s))", $3.code, $6.code, $10.code);  
    $$code = gen_code(temp);  
}  
;
```

// Definición de la sentencia if sin else

if_solo:

```
IF '(' expresion ')' '{' bloque_sentencias '}' {  
    sprintf(temp, "(if %s\n(progn %s))", $3.code, $6.code);  
    $$code = gen_code(temp);  
}
```

```
}  
;
```

// Definición de la declaración de variables

declaracion_variable:

```
INTEGER lista_declaracion {  
    $$code = $2.code;  
}  
;
```

// Definición de la lista de declaración de variables

lista_declaracion:

```
IDENTIF {  
    anadir_var_local($1.code);  
    sprintf(temp, "(setq %s_%s 0)", funcion_actual, $1.code);  
    $$code = gen_code(temp);  
}  
| IDENTIF '=' expresion {
```

```

    anadir_var_local($1.code);
    sprintf(temp, "(setq %s_%s %d)", funcion_actual, $1.code, $3.value);
    $$code = gen_code(temp);
}
| IDENTIF '[' expresion ']' {
    anadir_var_local($1.code);
    sprintf(temp, "(setq %s_%s (make-array %d))", funcion_actual, $1.code, $3.value);
    $$code = gen_code(temp);
}
| lista_declaracion ',' IDENTIF '=' expresion {
    anadir_var_local($3.code);
    sprintf(temp, "%s\n(setq %s_%s %d)", $1.code, funcion_actual, $3.code, $5.value);
    $$code = gen_code(temp);
}
| lista_declaracion ',' IDENTIF {
    anadir_var_local($3.code);
    sprintf(temp, "%s\n(setq %s_%s 0)", $1.code, funcion_actual, $3.code);
    $$code = gen_code(temp);
}

```

;

// Definición de la expresión

expresion:

termino {

 \$\$ = \$1;

}

| expresion '+' expresion {

 sprintf(temp, "(+ %s %s)", \$1.code, \$3.code);

 \$\$.code = gen_code(temp);

}

| expresion '-' expresion {

 sprintf(temp, "(- %s %s)", \$1.code, \$3.code);

 \$\$.code = gen_code(temp);

}

| expresion '*' expresion {

 sprintf(temp, "(* %s %s)", \$1.code, \$3.code);

 \$\$.code = gen_code(temp);

}


```
| expression '/' expression {  
    sprintf(temp, "(/ %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression '%' expression {  
    sprintf(temp, "(mod %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression '>' expression {  
    sprintf(temp, "(> %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression '<' expression {  
    sprintf(temp, "(< %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression IGUAL expression { sprintf(temp, "(= %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}
```

```
| expression DISTINTO expression {  
    sprintf(temp, "(/= %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression MENORIGUAL expression {  
    sprintf(temp, "(<= %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression MAYORIGUAL expression {  
    sprintf(temp, "(>= %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression AND expression {  
    sprintf(temp, "(and %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
  
| expression OR expression {  
    sprintf(temp, "(or %s %s)", $1.code, $3.code);  
    $$code = gen_code(temp);
```

```

    }

| '!' expresion %prec UNARY_SIGN {

    sprintf(temp, "(not %s)", $2.code);

    $$code = gen_code(temp);

}

| IDENTIF '(' argumentos ')' { // Llamada a función como expresión

    sprintf(temp, "(%s %s)", $1.code, $3.code);

    $$code = gen_code(temp);

}

;

```

// Definición de un término

termino:

```

operando {

    $$ = $1;

}

| '+' operando %prec UNARY_SIGN {

    $$ = $2;

}

```

```

| '-' operando %prec UNARY_SIGN {
    sprintf(temp, "(- %s)", $2.code);
    $$code = gen_code(temp);
}
;

```

// Definición de un operando

operando:

```

IDENTIF {
    if(funcion_actual == NULL || !es_variable_local($1.code)) {
        // Variable global
        sprintf(temp, "%s", $1.code);
    } else {
        // Variable local o parámetro
        sprintf(temp, "%s_%s", funcion_actual, $1.code);
    }
    $$code = gen_code(temp);
}
| IDENTIF '[' expresion ']' {

```

```

if(funcion_actual == NULL || !es_variable_local($1.code)) {
    // Variable global

    sprintf(temp, "(aref %s %s)", $1.code, $3.code);
} else {
    // Variable local o parámetro

    sprintf(temp, "(aref %s_%s %s)", funcion_actual, $1.code, $3.code);
}

$$code = gen_code(temp);
}

| NUMBER {
    sprintf(temp, "%d", $1.value);
    $$code = gen_code(temp);
}

| '(' expresion ')' {
    $$ = $2;
}

;

```

// Definición de la lista de elementos

lista_elementos:

```
    elemento {  
        $$code = $1.code;  
    }  
| lista_elementos ' ' elemento {  
    sprintf(temp, "%s\n%s", $1.code, $3.code);  
    $$code = gen_code(temp);  
}  
;
```

// Definición de un elemento

```
elemento: expresion {  
    sprintf(temp, "(princ %s)", $1.code);  
    $$code = gen_code(temp);  
}  
| STRING {  
    sprintf(temp, "(princ \"%s\")", $1.code);  
    $$code = gen_code(temp);  
}
```

```
;
```

```
// Definición de los argumentos de la función
```

```
argumentos:
```

```
/* vacío */ { $$code = gen_code(""); }
```

```
| lista_argumentos {
```

```
    $$code = $1.code;
```

```
}
```

```
;
```

```
// Definición de la lista de argumentos
```

```
lista_argumentos:
```

```
expresion {
```

```
    $$code = $1.code;
```

```
}
```

```
| lista_argumentos ',' expresion {
```

```
    sprintf(temp, "%s %s", $1.code, $3.code);
```

```
    $$code = gen_code(temp);
```

```
}
```

```
;
```

```
%% // SECCION 4 Codigo en C
```

```
int n_line = 1 ;
```

```
int yyerror (mensaje)
```

```
char *mensaje ;
```

```
{
```

```
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
```

```
    printf ( "\n" ) ; // bye
```

```
    return 0 ;
```

```
}
```

```
char *int_to_string (int n)
```

```
{
```

```
    sprintf (temp, "%d", n) ;
```

```
    return gen_code (temp) ;
```



```
}
```

```
char *char_to_string (char c)
```

```
{
```

```
    sprintf (temp, "%c", c);
```

```
    return gen_code (temp);
```

```
}
```

```
char *mi_malloc (int nbytes)    // reserva n bytes de memoria dinamica
```

```
{
```

```
    char *p ;
```

```
    static long int nb = 0;    // sirven para contabilizar la memoria
```

```
    static int nv = 0;        // solicitada en total
```

```
    p = malloc (nbytes);
```

```
    if (p == NULL) {
```

```
        fprintf (stderr, "No queda memoria para %d bytes mas\n", nbytes);
```

```
        fprintf (stderr, "Reservados %ld bytes en %d llamadas\n", nb, nv);
```

```
        exit (0);
```

```
}  
  
nb += (long) nbytes ;  
  
nv++ ;  
  
return p ;  
}
```

```
/*  
***** Seccion de Palabras Reservadas *****  
*/
```

```
typedef struct s_keyword { // para las palabras reservadas de C  
    char *name ;  
    int token ;  
} t_keyword ;
```

```
t_keyword keywords [] = { // define las palabras reservadas y los --- Se ponen las llaves para evitar warnings de compilacion
```

```

{"main",    MAIN},      // y los token asociados
{"int",     INTEGER},   // tipo de dato entero
{"if",      IF},        // sentencia if
{"else",    ELSE},      // sentencia else
{"while",   WHILE},     // sentencia while
{"puts",    PUTS},      // sentencia puts
{"printf",  PRINTF},    // sentencia printf
{"==",      IGUAL},     // igual
{"!=",      DISTINTO},  // distinto
{"<=",      MENORIGUAL}, // menor o igual
{">=",      MAYORIGUAL}, // mayor o igual
{"&&",      AND},       // and logico
{"||",      OR},        // or logico
{"for",     FOR},       // sentencia for
{"return",  RETURN},    // sentencia return
{NULL,     0}          // para marcar el fin de la tabla
};

```

```

t_keyword *search_keyword (char *symbol_name)

```

```
{          // Busca n_s en la tabla de pal. res.  
          // y devuelve puntero a registro (simbolo)  
  
    int i;  
    t_keyword *sim;  
  
    i = 0;  
    sim = keywords;  
    while (sim [i].name != NULL) {  
        if (strcmp (sim [i].name, symbol_name) == 0) {  
            // strcmp(a, b) devuelve == 0 si a==b  
  
            return &(sim [i]);  
        }  
        i++;  
    }  
  
    return NULL;  
}
```

```
/******  
***** Seccion del Analizador Lexicografico *****  
*****
```

```
char *gen_code (char *name)  // copia el argumento a un
```

```
{          // string en memoria dinamica
```

```
    char *p ;
```

```
    int l ;
```

```
    l = strlen (name)+1 ;
```

```
    p = (char *) mi_malloc (l) ;
```

```
    strcpy (p, name) ;
```

```
    return p ;
```

```
}
```

```
int yylex ()
```

```
{
```

```
// NO MODIFICAR ESTA FUNCION SIN PERMISO
```

```
int i ;
```

```
unsigned char c ;
```

```
unsigned char cc ;
```

```
char ops_expandibles [] = "!<=|>%&/+.*" ;
```

```
char temp_str [256] ;
```

```
t_keyword *symbol ;
```

```
do {
```

```
    c = getchar () ;
```

```
    if (c == '#') { // Ignora las lineas que empiezan por # (#define, #include)
```

```
        do {                //      OJO que puede funcionar mal si una linea contiene #
```

```
            c = getchar () ;
```

```
        } while (c != '\n') ;
```

```
    }
```

```
    if (c == '/') { // Si la linea contiene un / puede ser inicio de comentario
```

```
        cc = getchar () ;
```

```

if (cc != '/') { // Si el siguiente char es / es un comentario, pero...
    ungetc (cc, stdin);
} else {
    c = getchar (); // ...
    if (c == '@') { // Si es la secuencia //@ ==> transcribimos la linea
        do { // Se trata de codigo inline (Codigo embebido en C)
            c = getchar ();
            putchar (c);
        } while (c != '\n');
    } else { // ==> comentario, ignorar la linea
        while (c != '\n') {
            c = getchar ();
        }
    }
}
} else if (c == '\\') c = getchar ();

if (c == '\n')
    n_line++;

```

```
} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;
```

```
if (c == '\n') {
```

```
    i = 0 ;
```

```
    do {
```

```
        c = getchar () ;
```

```
        temp_str [i++] = c ;
```

```
    } while (c != '\n' && i < 255) ;
```

```
    if (i == 256) {
```

```
        printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line) ;
```

```
    } // habria que leer hasta el siguiente " , pero, y si falta?
```

```
    temp_str [--i] = '\0' ;
```

```
    yylval.code = gen_code (temp_str) ;
```

```
    return (STRING) ;
```

```
}
```

```
if (c == '.' || (c >= '0' && c <= '9')) {
```

```
    ungetc (c, stdin) ;
```



```

scanf ("%d", &yylval.value) ;

//    printf ("\nDEV: NUMBER %d\n", yylval.value) ;    // PARA DEPURAR

return NUMBER ;

}

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    i = 0 ;
    while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9') || c == '_') && i < 255) {
        temp_str [i++] = tolower (c) ;
        c = getchar () ;
    }
    temp_str [i] = '\0' ;
    ungetc (c, stdin) ;

    yylval.code = gen_code (temp_str) ;
    symbol = search_keyword (yylval.code) ;
    if (symbol == NULL) { // no es palabra reservada -> identificador antes vrvariable
//        printf ("\nDEV: IDENTIF %s\n", yylval.code) ; // PARA DEPURAR

```

```
        return (IDENTIF);
    } else {
//        printf ("\nDEV: OTRO %s\n", yylval.code);    // PARA DEPURAR
        return (symbol->token);
    }
}
```

```
if (strchr (ops_expandibles, c) != NULL) { // busca c en ops_expandibles
    cc = getchar ();
    sprintf (temp_str, "%c%c", (char) c, (char) cc);
    symbol = search_keyword (temp_str);
    if (symbol == NULL) {
        ungetc (cc, stdin);
        yylval.code = NULL;
        return (c);
    } else {
        yylval.code = gen_code (temp_str); // aunque no se use
        return (symbol->token);
    }
}
```

```
}
```

```
// printf ("\nDEV: LITERAL %d %%c#\n", (int) c, c); // PARA DEPURAR
```

```
if (c == EOF || c == 255 || c == 26) {
```

```
// printf ("tEOF "); // PARA DEPURAR
```

```
return (0);
```

```
}
```

```
return c;
```

```
}
```

```
int main() {
```

```
    yyparse();
```

```
    return 0;
```

```
}
```